

```
import pandas as pd

data = {
    'Word_Frequency': [
        2, 3, 1, 4, 2,
        8, 10, 12, 9, 15,
        3, 2, 5, 11, 14,
        1, 4, 7, 13, 16
    ],
    'Message_Length': [
        50, 60, 45, 70, 55,
        40, 35, 30, 45, 25,
        65, 80, 50, 35, 30,
        90, 75, 40, 28, 20
    ],
    'Spam': [
        0, 0, 0, 0, 0,
        1, 1, 1, 1, 1,
        0, 0, 0, 1, 1,
        0, 0, 1, 1, 1
    ]
}

df = pd.DataFrame(data)

df.to_csv("spam_svm.csv", index=False)

print("Dataset created successfully!")
print("Shape:", df.shape)
print(df)
```

```
Dataset created successfully!
Shape: (20, 3)
```

	Word_Frequency	Message_Length	Spam
0	2	50	0
1	3	60	0
2	1	45	0
3	4	70	0
4	2	55	0
5	8	40	1
6	10	35	1
7	12	30	1
8	9	45	1
9	15	25	1
10	3	65	0
11	2	80	0
12	5	50	0
13	11	35	1
14	14	30	1
15	1	90	0
16	4	75	0
17	7	40	1
18	13	28	1
19	16	20	1

```
data = pd.read_csv("spam_svm.csv")

print("Dataset:")
print(data)
```

```
Dataset:
```

	Word_Frequency	Message_Length	Spam
0	2	50	0
1	3	60	0
2	1	45	0
3	4	70	0
4	2	55	0
5	8	40	1
6	10	35	1
7	12	30	1
8	9	45	1
9	15	25	1
10	3	65	0
11	2	80	0
12	5	50	0
13	11	35	1
14	14	30	1
15	1	90	0
16	4	75	0
17	7	40	1
18	13	28	1
19	16	20	1

```
X = data[['Word_Frequency', 'Message_Length']]
y = data['Spam']

print("X:")
print(X)

print("\nY:")
print(y)
```

```
X:
   Word_Frequency  Message_Length
0                2                50
1                3                60
2                1                45
3                4                70
4                2                55
5                8                40
6               10                35
7               12                30
8                9                45
9               15                25
10               3                65
11               2                80
12               5                50
13              11                35
14              14                30
15               1                90
16               4                75
17               7                40
18              13                28
19              16                20
```

```
Y:
0    0
1    0
2    0
3    0
4    0
5    1
6    1
7    1
8    1
9    1
10   0
11   0
12   0
13   1
14   1
15   0
16   0
17   1
18   1
19   1
Name: Spam, dtype: int64
```

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=5
)
```

```
print("X_train:", x_train.shape)
print("Y_train:", y_train.shape)
print("X_test:", x_test.shape)
print("Y_test:", y_test.shape)
```

```
X_train: (16, 2)
Y_train: (16,)
X_test: (4, 2)
Y_test: (4,)
```

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()

x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

print("Data standardized successfully!")
```

```
Data standardized successfully!
```

```
from sklearn.svm import SVC

model = SVC(
    kernel='rbf',
    random_state=0
)

model.fit(x_train, y_train)

print("SVM model trained successfully!")
```

SVM model trained successfully!

```
svc_prediction = model.predict(x_test)

print("SVM Prediction:")
print(svc_prediction)

print("\nActual Values:")
print(y_test.values)
```

SVM Prediction:
[0 1 1 1]

Actual Values:
[0 1 1 1]

```
from sklearn import metrics

conf_mat = metrics.confusion_matrix(
    y_test,
    svc_prediction
)

print("Confusion Matrix :")
print(conf_mat)
```

Confusion Matrix :

```
[[1 0]
 [0 3]]
```

```
accuracy_score = metrics.accuracy_score(
    y_test,
    svc_prediction
)

print("Accuracy Score :", accuracy_score)
print("Accuracy in Percentage :", int(accuracy_score * 100), "%")
```

Accuracy Score : 1.0
Accuracy in Percentage : 100 %

```
print("Classification Report:")

print(
    metrics.classification_report(
        y_test,
        svc_prediction
    )
)
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	3
accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4

```
import seaborn as sn
import matplotlib.pyplot as plt

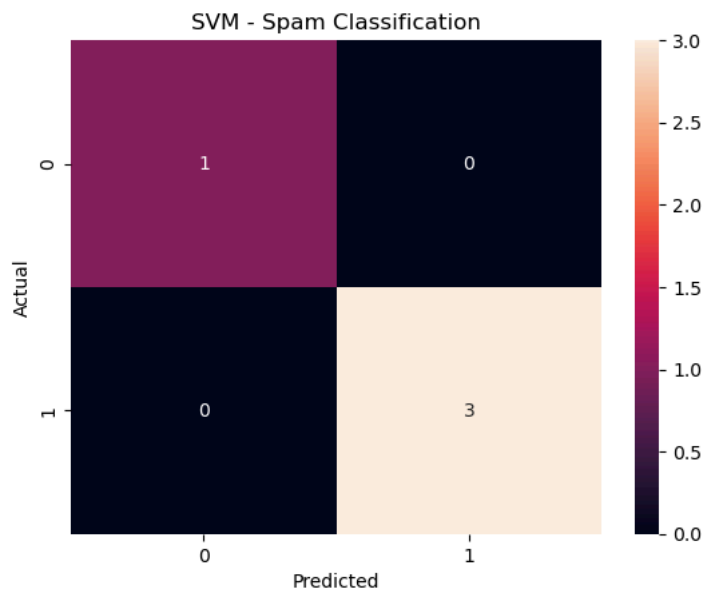
conf_mat = pd.crosstab(
    y_test,
    svc_prediction,
    rownames=['Actual'],
    colnames=['Predicted']
)
```

```

sn.heatmap(conf_mat, annot=True)

plt.title("SVM - Spam Classification")
plt.show()

```



```

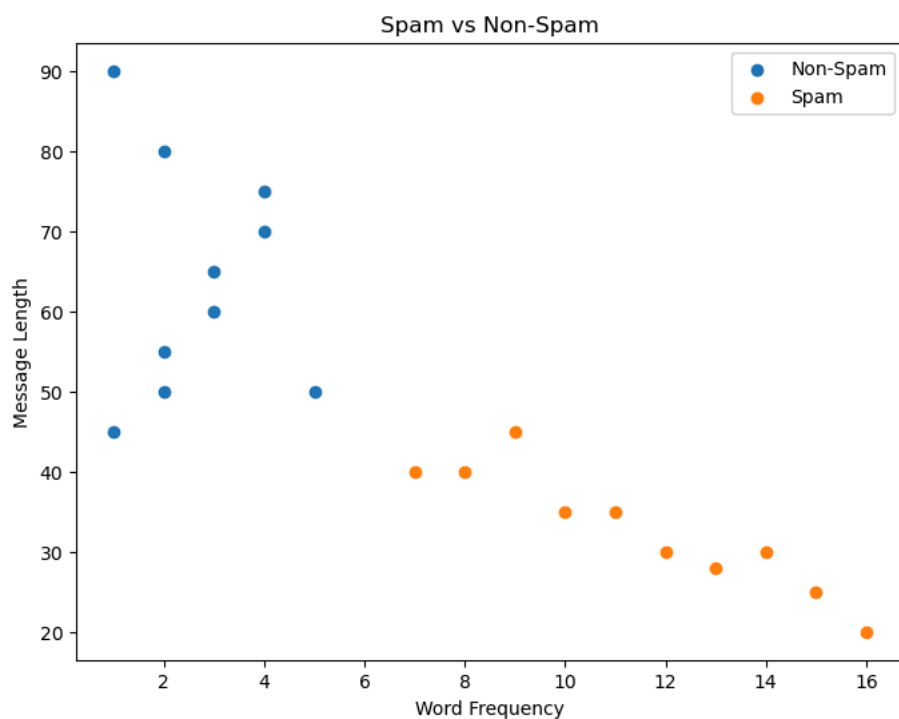
plt.figure(figsize=(8, 6))

plt.scatter(
    data[data['Spam'] == 0]['Word_Frequency'],
    data[data['Spam'] == 0]['Message_Length'],
    label='Non-Spam'
)

plt.scatter(
    data[data['Spam'] == 1]['Word_Frequency'],
    data[data['Spam'] == 1]['Message_Length'],
    label='Spam'
)

plt.xlabel("Word Frequency")
plt.ylabel("Message Length")
plt.title("Spam vs Non-Spam")
plt.legend()
plt.show()

```



Start coding or [generate](#) with AI.